Patterns & Tech Signals — How the System Works End to End

A guided overview of how this dashboard detects chart patterns, validates them against volume, and fuses every technical lens into a single confluence verdict. Everything below is drawn directly from the code in this repository, with the real thresholds, field names, and weights — not generic technical-analysis theory.

> **The one caveat that frames everything.** Chart patterns here are *deterministic geometric reads of recent price pivots* — one probabilistic input among many, never a prediction. The validation and confluence layers exist precisely so a hollow or contradicted pattern gets down-weighted instead of trusted blindly.

The pipeline has four stages, each implemented as a pure (network-free, unit-testable) function so the math can be tested with a synthetic price series:

```
              ~2y daily OHLCV (cached, 1 fetch/symbol)
                              |
                              v
+-----------------------------------------------------------+
| 1. DETECTION   technical_signals_service.py               |
|    adaptive zig-zag -> pivots -> named pattern (1/symbol)|
|    each tagged forming | confirmed + key level + target  |
+-----------------------------------------------------------+
                              |  detected pattern(s)
                              v
+-----------------------------------------------------------+
| 2. VALIDATION  risk_service.py  (+ volume_service.py)     |
|    0-1 volume score -> verdict:                           |
|    confirmed | weak | pending | veto | stale             |
+-----------------------------------------------------------+
                              |  pattern + validation block
                              v
+-----------------------------------------------------------+
| 3. CONFLUENCE  confluence_service.py                      |
|    5 weighted votes (trend, structure, momentum,          |
|    pattern, volume) -> bias + score100 + strength + tally|
+-----------------------------------------------------------+
                              |  confluence block
                              v
+-----------------------------------------------------------+
| 4. UI          dashboard.html + screening_service.py      |
|    Tech Stance . Patterns & Tech Signals table .          |
|    Confirmed-Patterns filter . Inspector cards            |
+-----------------------------------------------------------+
```

Section 1 — Pattern detection

Where: `services/technical_signals_service.py` (`compute_signals` / `compute_chart` → `_zigzag`, `_detect_patterns`).

Input window and the zig-ag

All signals come from roughly **2 years of daily history** (`TECHNICAL_SIGNALS_PERIOD`, default `2y`), fetched once per symbol and cached. A series needs **≥ 30 closes** to compute anything.

Patterns are matched on **swing pivots** found by an **adaptive, volatility-scaled zig-zag** (`_zigzag`). The reversal threshold is either a fixed percentage (`TECHNICAL_PIVOT_PCT`, default `0` = adaptive) or derived from volatility:

```
threshold% = clamp( ATR%  ×  2.5 ,  min 4% … max 18% )
```

- `2.5` = `TECHNICAL_PIVOT_ATR_MULT`, clamp edges = `TECHNICAL_PIVOT_MIN_PCT` (4) and `TECHNICAL_PIVOT_MAX_PCT` (18). If ATR% is unavailable it falls back to `max(min_pct, 6%)`.
- A larger threshold on a jumpy stock keeps the zig-zag from carving noise into fake pivots; a tight one on a calm stock still finds real swings.
- The zig-zag always **appends the current running extreme** as a tentative trailing pivot, so the live leg is represented — which is why detection scans a few recent windows, not just the very last pivots.

Pattern types

Each matcher reads a short run of alternating high/low pivots. "Similar" pivots are compared with a tolerance `TECHNICAL_PATTERN_TOL_PCT` (default **3%**; `tol = max(0.5%, tol_pct/100)`).

Pattern	Code	Direction	Pivot shape	Key level	Confirms when	Measured target
Double Top	`DT`	bearish	High → Low → High, tops similar	`neckline` = the trough between the tops	close **below** neckline	`neckline − (top − neckline)`
Double Bottom	`DB`	bullish	Low → High → Low, bottoms similar	`neckline` = the peak between the bottoms	close **above** neckline	`neckline + (neckline − bottom)`
Head & Shoulders	`H&S`	bearish	H-L-H-L-H, head highest, shoulders similar	`neckline` = average of the two troughs	close **below** neckline	`neckline − (head − neckline)`
Inverse H&S	`iH&S`	bullish	L-H-L-H-L, head lowest, shoulders similar	`neckline` = average of the two peaks	close **above** neckline	`neckline + (neckline − head)`
Ascending Triangle	`Asc△`	bullish	Flat highs (**≥3 touches**) + rising lows	`resistance` = avg of highs	close **above** the highs	*none*
Descending Triangle	`Desc△`	bearish	Flat lows (**≥3 touches**) + falling highs	`support` = avg of lows	close **below** the lows	*none*
Symmetrical Triangle	`Sym△`	neutral	Falling highs **and** rising lows converging	`apex` = midpoint of latest high/low	(always `forming`)	*none*

Selection priority — one pattern per symbol

A single swing can trivially contain several overlapping shapes (an H&S contains an inner "double bottom"; an ascending triangle's flat highs look like a "double top"). To avoid contradictory labels, `_detect_patterns` returns **one best read** ordered by structural completeness:

1. **Head & Shoulders first** — strongest, uses **5 pivots** (scanned over the last 6-pivot lookback). If found, it wins outright.
2. **Triangle vs Double** — a triangle (evaluated over the last 7 pivots) requires **≥3 touches on the flat side**, which disambiguates it from a 2-touch double (3 pivots, scanned over the last 5). **A triangle wins over a double** when both fire on the same swing, because it explains the flat side better.

So the precedence is **H&S (5 pivots) > triangle (≥3 flat-side touches) > double (3 pivots)**.

Forming vs confirmed

Every pattern carries a **status**:

Status	Meaning
`forming`	The shape is in place but price has **not yet broken the key level** in the pattern's direction. A *watch item*, not a trigger.
`confirmed`	Price has **broken through the key level** (neckline / support / resistance) in the pattern's direction. The measured-move **target becomes active**.

Example: a **Double Top** stays `forming` while price is above its neckline and only flips to `confirmed` once price is below it — which is why a stock near its highs can show "Double Top · forming" without that being a sell signal yet.

Confidence (the shape `%`)

Confidence rewards how *clean / textbook* the shape is — specifically how closely the two defining pivots match (the two tops, the two shoulders):

```
confidence = min(0.95, base + 0.35 × closeness)
```

- `base`: Head & Shoulders **0.60**, Double Top/Bottom **0.55**, Triangles **0.50**.
- `closeness`: 1.0 when the paired pivots are identical, decaying as they diverge.
- Capped at **0.95** — the model never claims certainty.

A higher % means a more textbook shape, **not** a higher probability of playing out.

Section 2 — Pattern validation (the volume validator)

Where: `services/risk_service.py` (`validate_pattern` / `validate_patterns`), using volume analytics from `services/volume_service.py`.

A pattern's *shape* says nothing about whether real trading conviction backs it. The **Risk agent** cross-checks each detected pattern against volume and attaches a `validation` block with a 0–1 **score** and a **verdict**. It is on by default (`ASSESSMENT_PATTERN_VOLUME=1`).

The four inputs to the score

The score **starts at 0.5** and is nudged up or down by four checks, then clamped to `[0, 1]`:

#	Check	What it looks at	Effect on score
1	**Breakout RVOL** *(only if `confirmed`)*	Peak relative volume from the pattern's last pivot through today (`VOLUME_BREAKOUT_RVOL`, default **1.3**). RVOL = day volume ÷ its 20-day average.	≥ 1.3× → **+0.30**; ≥ 1.0× → **+0.10**; < 1.0× → **−0.25**. If still `forming`: no breakout check, noted as "confirmation pending".
2	**Key-level conviction**	The reversal extreme (the **low** for a bullish pattern, the **high** for a bearish one) vs the **Point of Control (POC)** volume node it sits on.	High-volume node (≥70% of POC) → **+0.20**; medium (≥35%) → **+0.05**; low (<35%) → **−0.28**.
3	**OBV alignment**	On-Balance-Volume slope over ~21 bars vs the pattern's bias (accumulation should back a bullish pattern, distribution a bearish one).	Aligned → **+0.15**; diverges → **−0.15**.
4	**Triangle contraction**	For triangles only: volume should *coil* (decline) into the apex.	Contracting → **+0.10**; not contracting → noted only (no penalty).

The motivating example baked into the code: *a double bottom at $95 where only ~30% of POC volume traded* is a weak demand zone → it loses 0.28 on check #2 and is likely **vetoed**.

> **POC caveat.** A true tick-level Point of Control needs intraday volume-at-price no free feed provides. `volume_service.py` approximates it by spreading each day's volume across that day's High–Low range into 24 fixed bins over the last 252 sessions (`VOLUME_PROFILE_LOOKBACK`). Node labels: `high` ≥70% of POC, `medium` ≥35%, `low` <35%, `gap` = price outside the profiled range.

The four verdicts

After scoring, the verdict is assigned in this order (staleness can override):

Verdict	Glyph	Exact trigger	Meaning to the reader
`veto`	×	`score < 0.40` (`RISK_PATTERN_VETO_SCORE`) — checked **first**, even while forming	Conviction so poor the setup is rejected (e.g. a double bottom on a low-volume floor).
`pending`	·	`score ≥ 0.40` **and** status is **not** `confirmed`	Still forming — volume confirmation isn't possible until it breaks out.

`confirmed`	✓	status `confirmed` **and** `score ≥ 0.62` (`RISK_PATTERN_CONFIRM_SCORE`)	Broke out on expanding volume with supporting conviction.
`weak`	!	status `confirmed` **and** `0.40 ≤ score < 0.62`	Broke out, but the volume/conviction is unconvincing.
`stale`	⊘	**Overrides any of the above** when the pattern is played out or aged-and-departed (see below).	Played out or no longer describes the current structure.

Staleness override (`_staleness`) sets `stale` when **either**:

- **Played out** — price has reached/passed the measured-move target: `(price − key) / (target − key) ≥ 1.0` (`RISK_PATTERN_TARGET_DONE`). *(The ratio works for bearish patterns too, since both numerator and denominator are negative as the move progresses.)*
- **Aged & departed** — the pattern's last pivot is older than **90 sessions** (`RISK_PATTERN_STALE_AGE_BARS`) **and** price has moved more than **12%** (`RISK_PATTERN_STALE_MOVE_PCT`) away from the key level.

What the glyphs / badges mean in the UI

`dashboard.html` renders a mark next to each pattern (`renderFibValidationMark`) and a badge in the Inspector (`PATTERN_VALIDATION_META`):

Verdict	Mark	Badge label
`confirmed`	✓	"volume-confirmed"
`weak`	!	"weak volume"
`veto`	×	"volume veto"
`pending`	·	"confirmation pending"
`stale`	⊘	"played out / stale"

How the verdict re-weights the pattern

The validator also stores an `adjustedConfidence` = shape confidence × a per-verdict factor, so weak/veto/stale patterns are quietly de-emphasised everywhere:

Verdict	Confidence factor
`confirmed`	× 1.0
`pending`	× 0.85
`weak`	× 0.7
`veto`	× 0.4
`stale`	× 0.25

`RISK_PATTERN_ACTION` controls the policy: `downgrade` (default) keeps weak/veto patterns visible but down-weighted; `veto` drops veto- and stale-grade patterns entirely so they can't drive a recommendation.

Section 3 — The Confluence agent

Where: `services/confluence_service.py` (`compute_confluence`).

No single lens is reliable on its own, so the Confluence agent casts each technical read as a **weighted directional vote**, aggregates them into a normalised score and bias, and — crucially — produces explicit **agreement** and **conflict** lists so a recommendation can reason "trend, structure and a confirmed pattern all point up; only light volume dissents."

The five voting lenses and their weights

Lens	Reads	Base weight (env)
Trend	MA stack (20/50/200), golden/death cross, 3-month slope	`CONFLUENCE_WEIGHT_TREND` = **1.0**
Structure	zig-zag higher-highs / lower-lows read	`CONFLUENCE_WEIGHT_STRUCTURE` = **0.8**
Momentum	MACD histogram state / RSI(14) regime	`CONFLUENCE_WEIGHT_MOMENTUM` = **0.5**
Pattern	the Risk-validated pattern (verdict-weighted)	`CONFLUENCE_WEIGHT_PATTERN` = **1.0**
Volume	RVOL regime + OBV accumulation/distribution	`CONFLUENCE_WEIGHT_VOLUME` = **0.6**

Each vote's final weight = `base weight × magnitude`, where magnitude reflects how emphatic that lens is:

- **Trend:** MA stack bullish/bearish → direction ±1, magnitude **0.9**; otherwise a strong slope (`>+10%` / `<−10%` per year) → magnitude **0.45**. A matching golden/death cross adds **+0.1**.
- **Structure:** confirmed up/down-trend → ±1 @ **0.9**; "rising lows" / "falling highs" → ±1 @ **0.5**.
- **Momentum:** MACD state bullish/bearish → ±1 @ **0.6**; otherwise RSI ≥55 / ≤45 → ±1 @ **0.4**.
- **Pattern:** for each directional pattern, vote contribution = `direction × verdict_factor × confidence`, where `confidence` is the `adjustedConfidence` (fallback 0.5) and the **verdict factor** is:

Verdict	Pattern-vote factor
`confirmed`	1.0
`weak`	0.6
`pending`	0.45
`veto`	0.1
`stale`	0.05

The pattern lens weight is `1.0 × min(1.0, Σ contributions)` — a single strong confirmed pattern already saturates it. - **Volume:** OBV slope `>+1` → bullish, `<−1` → bearish; weight = `0.6 × state_weight`, where state weight is surging **1.0**, elevated **0.8**, normal **0.5**, light **0.3**.

From votes to bias, score and strength

```
net   = Σ (vote.sign × vote.weight)          # sign ∈ {+1, 0, −1}
score = clamp( net / Σ vote.weight ,  −1 … +1 )
score100 = round( (score + 1) / 2 × 100 )    # 0 = bearish, 50 = neutral, 100 = bullish
```

The score maps to a **bias** (band edges from `CONFLUENCE_LEAN_SCORE` = 0.15 and `CONFLUENCE_STRONG_SCORE` = 0.45):

Bias	Score band	UI class
`Bullish`	≥ **+0.45**	`bull`
`Lean Bullish`	**+0.15** … +0.45	`lean-bull`
`Mixed`	**−0.15** … +0.15	`mixed`
`Lean Bearish`	−0.45 … **−0.15**	`lean-bear`
`Bearish`	≤ **−0.45**	`bear`

Agreement vs conflict: votes that share the bias's sign are *agreements*, the opposite-sign ones are *conflicts*. The agent reports `agreeCount`, `conflictCount`, and `totalSignals` (the number of voting lenses).

Strength combines how far the score is from neutral with how unanimous the lenses are:

```
alignment  = agree_weight / directional_weight
conviction = |score| × 0.6  +  alignment × 0.4
```

Strength	Condition
`strong`	`conviction ≥ 0.66` **and** at least 2 directional votes
`moderate`	`conviction ≥ 0.40`
`weak`	otherwise

The block returned includes `bias`, `score`, `score100`, `strength`, `alignment`, `agreeCount`, `conflictCount`, `totalSignals`, the per-lens `votes`, and human-readable `agreements` / `conflicts` / `summary` / `message`. The same `bias` is also exposed as `stance`, which replaces the Fib-only stance in the tables when present.

Section 4 — How it surfaces in the UI

Where: `dashboard.html` (+ `services/screening_service.py` assembling rows).

Screening "Tech Stance"

The **Tech Stance** column on both the **Screening** and **Patterns & Tech Signals** tabs is driven by the **Confluence bias** (`renderTechStanceCell`). When a confluence verdict exists it shows the fused bias, a compact score meter (`score100`), and `strength · agreeCount/totalSignals`. It **falls back** to the Fib-position

stance (`Strong` / `Alert` / `Cautious` / `Neutral` / `Unknown`) only when there isn't enough history to fuse (`_apply_confluence_stance` / `_confluence_summary`).

Patterns & Tech Signals table columns

The `fibColumns` table (`dashboard.html`) shows, per symbol:

Column	Content
Symbol	Links into the Inspector
Price	Latest close
Rel. Vol	RVOL × and state (light / normal / elevated / surging)
Pattern	Badge `◆ CODE status (conf%)` **+ the validation glyph** (✓ ! ✕ · ⊘)
Neckline	The pattern's key level
Target	Measured-move target (green bullish / red bearish; blank for triangles)
Trends	Count of zig-zag legs, e.g. `5 Legs (3 ↑, 2 ↓)`
Tech Stance	Confluence bias + mini score meter (fallback Fib stance)
Nearest Fib	Closest Fibonacci level, kind, and price
Distance	% distance to that level

Confirmed-Patterns filter

A **"Confirmed Patterns"** toggle (`data-fib-confirmed` / `toggleFibConfirmed`) restricts the table to symbols whose chart pattern is `confirmed` — i.e. price has broken the key level in the pattern's direction and the measured-move target is active.

Inspector cards

The Inspector pairs a **Chart Patterns** panel and a **Volume & Liquidity** card (RVOL, OBV trend, POC, Value Area, price node) with a **Technical Confluence** card (`renderInspectorConfluence`): the bias chip, a score meter labelled "*0 = bearish, 50 = neutral, 100 = bullish*", per-lens vote chips (↑/↓/· per agent), and explicit **Agreeing** vs **Conflicting** lists with the `strength · N/N agree` summary. A `?` button opens the in-app **Patterns help** modal (`openPatternsHelp`) that documents all of the above.

End-to-end flow (recap)

```
daily OHLCV --> adaptive zig-zag pivots --> named pattern (1/symbol, forming|confirmed)
                |
                +--> volume_service: RVOL, OBV, volume profile / POC / nodes
                |
                v
  risk_service: 0.5 +/-(breakout RVOL, key-level node, OBV, triangle coil)
                --> score 0-1 --> verdict {veto<0.40 | pending | weak<0.62 | confirmed>=0.62},
```

```
                staleness override --> stale
            |
            v
  confluence_service: 5 weighted votes (trend 1.0, structure 0.8, momentum 0.5,
                pattern 1.0 x verdict-factor, volume 0.6)
            --> score (-1..+1) --> score100 --> bias (Bullish..Bearish) + strength + agree/total
            |
            v
  dashboard.html: Tech Stance . Patterns table (+ verdict glyphs) .
                Confirmed-Patterns filter . Inspector Confluence card
```

Key thresholds at a glance

Constant (env var)	Default	Role
`TECHNICAL_SIGNALS_PERIOD`	`2y`	History window for patterns/trends
`TECHNICAL_PIVOT_ATR_MULT` / `_MIN_PCT` / `_MAX_PCT`	2.5 / 4 / 18	Adaptive zig-zag threshold = clamp(ATR%×2.5, 4–18%)
`TECHNICAL_PATTERN_TOL_PCT`	3	How close pivots must be to count as "similar"
Triangle flat-side touches	≥ 3	Distinguishes a triangle from a 2-touch double
Confidence base / cap	H&S 0.60, double 0.55, triangle 0.50 / cap 0.95	`min(0.95, base + 0.35×closeness)`
`VOLUME_BREAKOUT_RVOL`	1.3	RVOL on the break to count as volume-confirmed
`RISK_PATTERN_CONFIRM_SCORE`	0.62	Score at/above which a confirmed break is `confirmed` (else `weak`)
`RISK_PATTERN_VETO_SCORE`	0.40	Score below which a pattern is `veto` (checked first)
Validation score start	0.50	±0.30/0.10/−0.25 breakout · +0.20/0.05/−0.28 node · ±0.15 OBV · +0.10 triangle
`RISK_PATTERN_STALE_AGE_BARS` / `_STALE_MOVE_PCT` / `_TARGET_DONE`	90 / 12% / 1.0	Staleness: aged-and-departed, or measured move complete
`CONFLUENCE_WEIGHT_*` (trend/struct/mom/pattern/vol)	1.0 / 0.8 / 0.5 / 1.0 / 0.6	Per-lens confluence vote weights
Verdict→pattern-vote factor	confirmed 1.0, weak 0.6, pending 0.45, veto 0.1, stale 0.05	Down-weights unvalidated patterns in confluence
`CONFLUENCE_LEAN_SCORE` / `_STRONG_SCORE`	0.15 / 0.45	Bias band edges
Strength	conviction ≥0.66 (≥2 dir.) strong, ≥0.40 moderate, else weak	`abs(score)×0.6 + alignment×0.4`

Implementation: `services/technical_signals_service.py` *(detection),* `services/volume_service.py` *(RVOL / OBV / profile / POC),* `services/risk_service.py` *(validation + staleness),* `services/confluence_service.py` *(fused verdict),* `services/screening_service.py` *(row assembly), and* `dashboard.html` *(table, glyphs, Inspector cards, help modal).*

Q&A — How it fits together (worked example: DFRYF)

The four questions below come up most often when reading a live card. They're answered against a real dashboard example — **DFRYF / Avolta AG**, which showed:

> **Technical Confluence:** *Lean Bullish · Moderate · 2/5 Agree.* Agreeing (2): Structure (rising lows); Momentum (MACD bullish, RSI 62). Conflicting (1): Pattern — Head & Shoulders (veto). Footnote: "*Trading at $63.14, comfortably above 50% technical support level ($60.38). Highly bullish trend baseline.*"

> **Chart Patterns:** Head & Shoulders, **VOLUME VETO**, `forming`, 94% confidence, neckline $56.77, target $48.66. Reasons: "*Not yet broken out — volume confirmation pending*", "*Key level in a low-volume zone (0% of POC) — weak supply*", "*OBV confirms (distribution)*". Volume: **REL VOL 18.18× surging**, OBV **Distribution**, POC $57.65.

Q1 — Is a pattern's verdict decided purely by volume? How does it relate to the `forming → confirmed` shape status?

There are **two independent things**, and conflating them is the usual source of confusion:

- **Shape status (`forming → confirmed`) is pure price geometry — zero volume.** Decided inside the matchers in `technical_signals_service.py` purely by where price sits versus the key level. For the H&S (`_match_hs`): `confirmed` only if `price < neckline`, else `forming`. DFRYF's H&S is *bearish* and confirms on a close **below** $56.77; price is $63.14 → `forming`.
- **Validation verdict is decided** *solely* **by four volume checks + the stale override** (`risk_service.py::validate_pattern`). The score starts at **0.50**:

Check	Δ score
1. Breakout RVOL — **only if shape status is** `confirmed`	≥1.3 → +0.30; ≥1.0 → +0.10; else −0.25
2. Key-level POC node	high(≥70%) +0.20; medium(≥35%) +0.05; low(<35%) **−0.28**
3. OBV alignment with bias	aligned +0.15; diverges −0.15
4. Triangle contraction (triangles only)	+0.10 / none

Shape confidence is **not** an input to the score (it's only used afterward for `adjustedConfidence`). Verdict order: `score < 0.40 → veto` (checked first); else `status != confirmed → pending`; else `confirmed` if `score ≥ 0.62` else `weak`; a `stale` override can replace any of these.

How shape-status *gates* **the verdict:** check #1 is skipped entirely unless the shape is already `confirmed`, and a non-vetoed `forming` pattern can only reach `pending`. So a pattern must break the key level (geometry) *before* volume can ever promote it to `confirmed`/`weak`.

DFRYF reconstruction: `forming` → breakout check skipped; key level at 0% of POC → **−0.28**; OBV negative (distribution) aligns with the *bearish* H&S → **+0.15**. Score = `0.5 − 0.28 + 0.15 = 0.37 < 0.40` → **veto** (and because veto is tested before the forming→pending branch, it never even becomes `pending`).

The "*volume confirmation pending*" bullet is just the reason string from check

1's "not confirmed" branch — it coexists with the veto.

Q2 — Is the UI "Tech Stance" the Confluence agent's fused output?

Yes. `renderTechStanceCell` (`dashboard.html`) shows `confluence.bias` + `strength` + `agreeCount/totalSignals` when a confluence verdict exists, and **falls back** to the Fib-position stance otherwise (row assembly in `screening_service._apply_confluence_stance`). The fusion is `confluence_service.compute_confluence` over five weighted lenses:

Lens	Base weight
Trend (MA stack / cross / slope)	**1.0**
Structure (zig-zag HH/LL)	**0.8**
Momentum (MACD / RSI)	**0.5**
Pattern (verdict-weighted)	**1.0**
Volume (OBV / RVOL state)	**0.6**

`score = clamp(Σ sign×weight / Σ weight, −1..+1)`; bias bands at ±0.15 / ±0.45; `strength` from `conviction = |score|×0.6 + alignment×0.4`.

DFRYF maps exactly to "Lean Bullish · Moderate · 2/5 Agree":

Lens	Direction	Weight	Role
Structure (rising lows)	+1 @ 0.5	0.40	agree
Momentum (MACD bullish)	+1 @ 0.6	0.30	agree
Pattern (H&S **veto**)	−1	≈0.038	conflict
Trend	**0 (neutral)**	≈0.30	—
Volume	**0 (neutral)**	≈0.60	—

`net ≈ 0.40 + 0.30 − 0.038 = 0.662`; `Σweight ≈ 1.64`; `score ≈ 0.404` → **Lean Bullish** (≥0.15, <0.45); `score100 ≈ 70`; `alignment ≈ 0.95`; `conviction ≈ 0.62` → **moderate**; agree 2, conflict 1, total 5. Note the **two neutral lenses** (Trend, Volume) — that's why 2 agree + 1 conflict ≠ 5, and their weight still sits in the denominator (see Q4).

Q3 — Do Fibonacci levels feed the Confluence agent?

No. `compute_confluence` only reads `trend`, `swing`, `momentum`, `patterns`, `volume`, `volumeProfile` — there is no Fib input in any lens. The **Trend lens** (`_trend_vote`) uses only the SMA 20/50/200 stack, the 50/200 golden/death cross, and a 63-day slope — **no Fib**.

Fibonacci levels feed only: (i) the Inspector **chart lines**, (ii) the **Nearest Fib** table column, and (iii) the **fallback Fib stance** (`inspector_service.build_technical_advisory`). That fallback is used as Tech Stance only when confluence is absent.

The DFRYF footnote "*...comfortably above 50% technical support level ($60.38). Highly bullish trend baseline.*" is **verbatim Fib-advisory prose** from `build_technical_advisory` (the `Strong` branch), surfaced in the Confluence card only as the `confluence-fib-note`. The words "trend baseline" are Fib-stance wording — independent of the actual Trend lens, which for DFRYF voted **neutral**.

Q4 — Could we synthesize a chained "what flips the verdict" explanation?

Feasible, and ~70% of the data already exists — `votes` (with `sign`, `weight`, `label`), `agreements`/`conflicts`, plus per-pattern `validation` (`score`, `reasons`, `keyLevelNode.pctOfPoc`, `breakoutRvol`, `obvSlopePct`, `staleness`). The two gaps: **(a)** the per-sub-check *signed deltas* (−0.28, +0.15…) are computed then discarded — only the aggregate score + reason strings survive; **(b)** there is no *counterfactual re-score* of "what bias results if lens X resolves."

The critical correction (why the intuitive story is backwards for DFRYF):

- The H&S is **bearish**. Vetoed, it barely counts (factor 0.1 → weight ≈0.038). If the veto "cleared" by the pattern actually **confirming** (a close below $56.77 on volume), it becomes a **confirmed bearish** vote at near-full weight (≈0.94): `net ≈ 0.70 − 0.94 = −0.24` → `score ≈ −0.09` → **Mixed**, i.e. *less* bullish. So "resolve the veto → more bullish" is **wrong** here.
- **The real limiter is the neutral, high-weight Volume lens.** The bearish pattern only subtracts ≈0.038; what actually keeps the score below the 0.45 "Bullish" edge is the denominator inflation from the two neutral lenses (Volume ≈0.60 + Trend ≈0.30). Flip **Volume** bullish (OBV slope > +1 → accumulation) and `score ≈ 1.26/1.64 ≈ 0.77` → **Bullish (~88)**.
- **"18.18× surging RVOL" is the current-bar** `volume_block.rvol` (the UI's "Rel Vol"), a *different quantity* from the `_breakout_rvol` used in check #1 — and that check is skipped while the pattern is `forming`. So the surge contributes nothing to the verdict; the chain must cite **specific** preconditions ("a confirmed break of the $56.77 neckline on ≥1.3× volume", "the neckline zone becoming a ≥35–70% POC node", "OBV slope > +1"), never the vague word "volume".

> **Status:** this is now being implemented as a concise **"Watch: …"** synthesis line in the Tech Stance / Confluence card — driven by a *signed numeric counterfactual* over the actual vote weights and the **specific failing sub-check(s)**, rather than verbal heuristics, precisely because DFRYF shows the naïve "clear the veto" framing would mislead.